

Network Programming:

Clients and servers are applications or processes that can run locally on a single computer or remotely across a network of computers. The resources required for this type of application are IP addresses, Port and sockets.

IP address:

Every computer on a network has a unique identifier called an **IP address** (IP stands for Internet Protocol). This address can be specified either as an **IP number** or as an **IP name**. An IP number typically has the form **ddd.ddd.ddd.ddd**, where each *d* is a digit. Because IP numbers are difficult to remember, so people generally use an IP name to specify an IP address.

Java represents an IP address with the class **InetAddress**, as defined in the package `java.net`. This class includes a static method that allows the programmer to obtain the IP address of the local host (the computer on which the program is currently running), as demonstrated by the next program:

// Displays IP address of current host

```
import java.net.*;

public class HostInfo{
    public static void main(String[] args){
        try{
            InetAddress ipAddress = InetAddress.getLocalHost();
            System.out.println("IP address:\n" + ipAddress);
        }catch(UnknownHostException e){
            System.out.println("Unknown host:\n" + e.toString());
        } } }
```

When this program is run on the computer, the output is:

```
IP address:
rashmi/192.168.1.2
```

Note that the call of the `getLocalHost` method is embedded in a try-catch statement. If the host computer does not happen to have an IP address, the catch clause displays the information about the unknown host exception.

Now suppose we know a host's IP name but want to obtain the complete IP information, including the IP number. The next program uses the static method `getByName` to accomplish this:

// Get the IP address of any machine by name

```
import java.net.*;
import java.util.Scanner;

public class Resolver{
    public static void main(String args[]){
        System.out.print("Name: ");
        Scanner reader = new Scanner(System.in);
```

```

try{
String name = reader.next();
InetAddress ipAddress = InetAddress.getByName(name);
System.out.println("IP address:\n" + ipAddress);
}
catch (Exception e){
System.out.println("Unknown host:\n" + e.toString());
}}

```

If the user enters the IP name `www.nast.edu`, the output will be `www.nast.edu/140.160.244.16`. Unlike the method `getLocalHost`, the method `getByName` can search the entire Internet for the IP address of the given name.

Understanding Port:

Clients connect to servers via objects known as **ports**. A port serves as a channel through which several clients can exchange data with the same server or with different servers. Ports are usually specified by numbers. Some ports are dedicated to special servers or tasks. For example, many computers reserve port number 13 for the day/time server, which allows clients to obtain the date and time. Port number 80 is reserved for a Web server, and so forth. Most computers also have hundreds or even thousands of free ports available for use by network applications.

Let us now illustrate some of the fundamental aspects of using ports, servers, and clients. We do this in the context of a day/time service involving two programs. The first program is a server listening on port 5555 for a request from a client. The second program is a client that connects to port 5555. We must run the server program first, and we can run the client program any time thereafter.

Introduction to Socket:

Socket is an object which establishes a communication link between two ports. The working mechanism of the socket is as shown in the figure below;

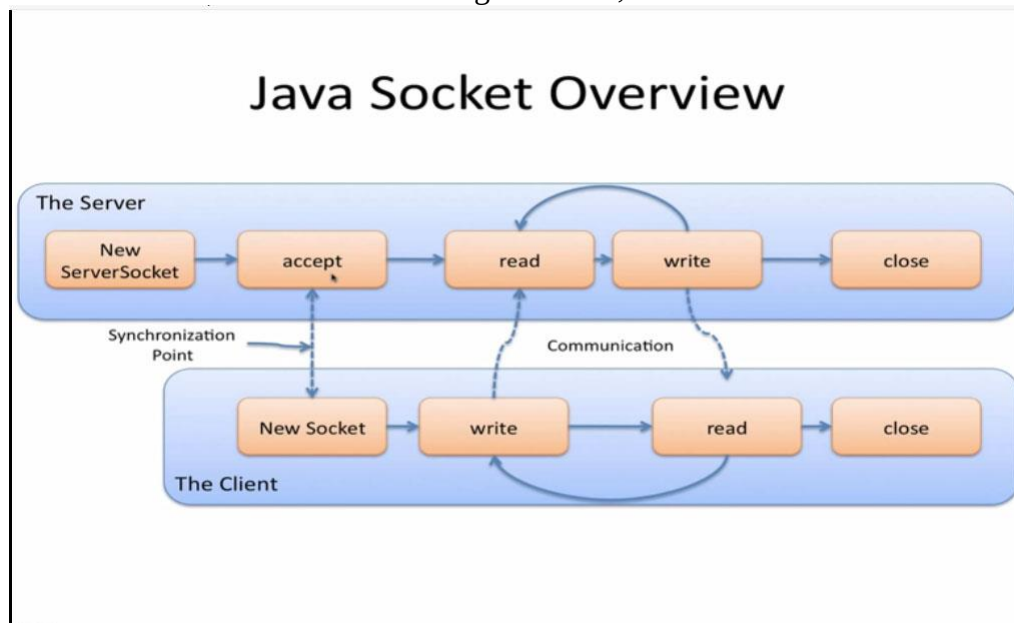


Fig: overview of java socket

Working mechanism of the socket is as shown in the table below by the example of the day/time server:

SN	Server Program	Client Program
1	Informs the user that it has started	
2	Sets up a server to listen for client on port; certain port say 5555	
3	Waits for a client to make a connection	
4		Asks the user for the IP address of the available server for example day/time server
5		After the user inputs the day/time server, connections on port 5555
6		Establishes the input stream on the connection
7	Establishes the output stream on the connection.	
8	Writes the day/time to the stream.	
9		Reads the day/time from the stream
10	Closes the connection	Closes the connection

Socket programming:

// A client program for socket programming

```

package javasocket;
import java.io.*;
import java.net.*;

public class javaSocketClient {
public static void main(String[] args){
    int portNumber = 5432;
    String hostname = "localhost";
    try{
        Socket echoSocket=new Socket(hostname,portNumber);
        System.out.println("client 1: connected to: " + echoSocket.toString());
        PrintWriter out=new PrintWriter(echoSocket.getOutputStream(),true);
        System.out.println("client 1: Do you want to send to the echo server");
        BufferedReader in = new BufferedReader(new
InputStreamReader(echoSocket.getInputStream()));
        BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));
        String userInput;
        while((userInput=stdIn.readLine())!=null){
            out.println(userInput);
            System.out.println("what do you want to send anything else:");
            System.out.println("echo:"+in.readLine());

```

```

    }
    }catch(IOException e){
        System.out.println("exception caught by client");
    }
}
}

```

// A server side program for socket programming

```

package javasocket;
import java.io.*;
import java.net.*;

public class JavaSocket {
    public static void main(String[] args) throws IOException {
        int portNumber = 5432;
        String serverIP = "localhost";
        try{
            ServerSocket serverSocket=new ServerSocket(portNumber) ;
            System.out.println("server started at:"+portNumber);
            Socket clientSocket=serverSocket.accept();
            System.out.println("got      connection      from"+clientSocket.getInetAddress()+"port
number:"+clientSocket.getPort());
            PrintWriter out=new PrintWriter(clientSocket.getOutputStream(),true);
            BufferedReader      in      =      new      BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
            String inputLine;
            while((inputLine=in.readLine())!=null){
                System.out.println("Got message:"+inputLine+"from the client");
                out.println(inputLine);
            }
        }catch(IOException e){
            System.out.println("exception caught by server");
        }
    }
}

```